

RAG (Retrieval Augmented Generation) Cheat Sheet

15 tables · 159 concepts · Updated 2026-09-27 · cheatgrid.com

Retrieval-Augmented Generation (RAG) is an LLM pattern that **retrieves external knowledge** at query time and injects it into the model's context. That grounding is how practitioners reduce hallucinations, bring in **fresh or private data**, and make outputs auditable through sources and citations. A useful mental model is that RAG is two coupled systems: an **information retrieval** system (indexes, rankers, filters) and a **response synthesis** system (prompting, citations, formatting). By 2026, production RAG has moved well beyond naive chunk-and-retrieve, and the dominant pattern is **agentic RAG**, where the LLM itself decides when, what and how to retrieve. Most "RAG problems" are retrieval problems first: if the right evidence never makes it into context, generation quality cannot recover it.

CONTENTS

RAG Building Blocks (Conceptual)	1
Document Loading and Parsing	2
Chunking and Splitting	3
Embedding Models	5
Dense Indexing and ANN	5
Vector Databases	6
Sparse and Hybrid Retrieval	7
Query Transformation	8
Reranking and Fusion	8
Query Engines and Answer Grounding	9
Advanced RAG Architectures	10
Storage, Persistence, and Caching	12
RAG Evaluation Metrics and Frameworks	12
Observability and Security	13
Multimodal RAG	14
References	15

TABLE 1

RAG Building Blocks (Conceptual)

Every RAG pipeline is assembled from the same handful of stages, run in order from ingesting raw data all the way to evaluating the final answer. Knowing this skeleton makes the rest of the cheat sheet easier to navigate: each later table zooms into one of these stages and lists the concrete tools you can plug in there.

Stage	Example	Description
Retrieval-Augmented Generation (RAG)	<pre>answer = LLM(question, context=top_k_docs)</pre>	Generates with retrieved evidence rather than relying only on parametric memory.
2-Step RAG vs Agentic RAG	<pre>docs = retriever.invoke(q) # 2-step: retrieve once, then generate answer = agent.invoke({"messages": [{"user", q}]}) # agentic: agent decides if/when to call the retriever tool</pre>	• 2-Step RAG always retrieves before generating, in one fixed pass • Agentic RAG lets an LLM agent decide when and how many times to retrieve

Stage	Example	Description
Ingestion	<code>docs = loader.load_data()</code>	Reads source data and converts it into document objects for downstream processing.
Chunking	<code>splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)</code>	Splits documents into smaller units so retrieval can target the right passages.
Embedding	<code>emb = client.embeddings.create(model="text-embedding-3-small", input="...")</code>	Maps text to vectors for similarity search in dense retrieval.
Indexing	<code>index.add(xb)</code>	Builds a search structure over vectors (exact or ANN) to enable fast retrieval.
Retrieval	<code>docs = retriever.invoke("...")</code>	Selects candidate chunks/documents relevant to the query.
Reranking	<code>reranked = co.rerank(query=q, documents=texts)</code>	Reorders retrieved candidates to improve top-k quality .
Synthesis	<code>resp = query_engine.query("...")</code>	Produces the final answer from retrieved context (often via an LLM).
Citations	<code>resp = citation_engine.query("...")</code>	Attaches sources to claims (usually at chunk-level granularity).
Evaluation	<code>score = faithfulness</code>	Measures retrieval + generation quality with task-appropriate metrics.

TABLE 2

Document Loading and Parsing

Getting raw files, pages and connector data into `Document` and `Node` objects with metadata attached, the step before chunking. LlamaIndex and LangChain both ship loader classes for common formats, plus pipelines that extract metadata and keep re-ingested documents deduplicated.

Concept	Example	Description
SimpleDirectoryReader	<pre>from llama_index.core import SimpleDirectoryReader reader = SimpleDirectoryReader(input_dir="path/to/directory") documents = reader.load_data()</pre>	Loads every supported file (PDF, DOCX, Markdown, images and more) from a local directory into <code>Document</code> objects, attaching file metadata automatically.
Document and Node	<pre>from llama_index.core import Document document = Document(text="text", metadata={"filename": "<doc_file_name>", "category": "<category>"},)</pre>	• A Document wraps a whole source (a PDF, an API response, a database row) with metadata. • A Node is a chunk of a Document, with its own metadata and links to neighboring nodes.
Document Loaders (LangChain)	<pre>from langchain_community.document_loaders import PyPDFLoader loader = PyPDFLoader(file_path="./example_data/layout-parser-paper.pdf") docs = loader.load()</pre>	• Every loader implements <code>load()</code> / <code>lazy_load()</code> and returns <code>Document</code> objects. • <code>PyPDFLoader</code> reads a PDF page by page; <code>WebBaseLoader</code> fetches one or more URLs and parses the HTML.
Data Connectors (LlamaHub Readers)	<pre>from llama_index.readers.notion import NotionPageReader reader = NotionPageReader(integration_token="secret_...") documents = reader.load_data(page_ids=["<page_id>"])</pre>	• Hundreds of community-published readers (Notion, Google Docs, Slack, SQL databases) load a source straight into <code>Document</code> objects • install only the <code>llama-index-readers-<source></code> package needed

Concept	Example	Description
IngestionPipeline	<pre>from llama_index.core.ingestion import IngestionPipeline from llama_index.core.node_parser import SentenceSplitter from llama_index.embeddings.huggingface import HuggingFaceEmbedding pipeline = IngestionPipeline(transformations=[SentenceSplitter(), HuggingFace Embedding(model_name="BAAI/bge-small-en-v1.5")]) nodes = pipeline.run(documents=documents)</pre>	- Chains transformations (splitting, embedding, metadata extraction) into one step - each node+transformation pair is cached so a repeated run skips unchanged work
Document Management (Upserts)	<pre>from llama_index.core.storage.docstore import SimpleDocumentStore from llama_index.core.ingestion import IngestionPipeline, DocstoreStrategy pipeline = IngestionPipeline(transformations=[SentenceSplitter()], docstore=SimpleDocumentStore(), docstore_strategy=DocstoreStrategy.UPSERTS,)</pre>	- Attaching a docstore lets the pipeline compare each document's hash and re-process only new or changed documents on re-ingest. UPSERTS_AND_DELETE also drops documents no longer present in the batch.
Metadata Extractors	<pre>from llama_index.core.extractors import SummaryExtractor, QuestionsAnsweredExtractor extractors = [SummaryExtractor(summaries=["prev", "self", "next"]), QuestionsAnsweredExtractor(questions=3),]</pre>	- LLM-driven extractors add context a bare chunk would lack: <code>TitleExtractor</code>, <code>SummaryExtractor</code>, <code>QuestionsAnsweredExtractor</code>, <code>EntityExtractor</code>. - Chain them with a node parser or inside an <code>IngestionPipeline</code>.
LlamaParse	<pre>from llama_cloud import LlamaCloud client = LlamaCloud(api_key="llx-...") job = client.parsing.create(tier="agentic", file_id="your-file-id")</pre>	Hosted, layout-aware parser from LlamaIndex that turns PDFs, Word, PowerPoint and Excel files into clean markdown, handling multi-column pages and tables that plain text extraction breaks apart.
Unstructured (partition_pdf)	<pre>from unstructured.partition.pdf import partition_pdf elements = partition_pdf(filename="example-docs/ layout-parser-paper-fast.pdf")</pre>	- Open-source library that partitions PDFs, Word docs, HTML and more into typed elements (title, table, narrative text) - its <code>hi_res</code> strategy switches on a layout detection model for scanned or multi-column pages

TABLE 3

Chunking and Splitting

How you slice documents into chunks quietly decides how good your retrieval can ever be: too big and you bury the answer in noise, too small and you lose the surrounding context. These splitters range from the dependable recursive-character default to semantic and structure-aware options, plus the parent/child, hierarchical, and late-chunking tricks that fix the boundary problems naive splitting creates.

Splitter	Example	Description
RecursiveCharacterTextSplitter	<pre>RecursiveCharacterTextSplitter(separators=["\n\n", "\n", " ", ""], chunk_size=512, chunk_overlap=50)</pre>	- Default general-purpose splitter that tries separators in order to form sized chunks - a 2026 benchmark across 50 academic papers found it the strongest single default, ahead of fixed-size and semantic chunking
CharacterTextSplitter	<pre>CharacterTextSplitter(separator= "\n\n", chunk_size=512, chunk_overlap=50)</pre>	Simple splitter using a fixed separator and target chunk size
TokenTextSplitter	<pre>TokenTextSplitter(chunk_size=512 , chunk_overlap=64)</pre>	Splits by token count (useful when you need predictable model-context usage)

Splitter	Example	Description
SentenceSplitter	<code>SentenceSplitter(chunk_size=512, chunk_overlap=50)</code>	Splits text into sentence-based chunks with size/overlap controls
MarkdownHeaderTextSplitter	<code>MarkdownHeaderTextSplitter(headers_to_split_on=[("#", "h1"), ("##", "h2")])</code>	- Splits Markdown while preserving header structure as metadata - chain a size-based splitter such as <code>RecursiveCharacterTextSplitter</code> afterward to cap any section that is still too long
HTMLHeaderTextSplitter	<code>HTMLHeaderTextSplitter(headers_to_split_on=[("h1", "h1"), ("h2", "h2")])</code>	Splits HTML by header tags to keep section semantics
CodeSplitter	<code>CodeSplitter(language="python", chunk_lines=40, chunk_lines_overlap=15, max_chars=1500)</code>	- Splits source code with an AST-aware parser so chunks respect function and class boundaries - supports both character-based and token-based chunk sizing across many languages
RecursiveJsonSplitter	<code>RecursiveJsonSplitter(max_chunk_size=300).split_json(json_data)</code>	Splits nested JSON by walking the object hierarchy , keeping related keys together instead of cutting mid-object
ParentDocumentRetriever	<code>ParentDocumentRetriever(vectorstore=vs, docstore=store, child_splitter=small_splitter)</code>	Indexes small child chunks for precise recall but returns the larger parent chunk to the LLM for richer context (now ships in <code>langchain-classic</code>)
SemanticChunker	<code>SemanticChunker(embeddings, breakpoint_threshold_type="percentile")</code>	- Splits at semantic boundary breaks in embedding space, with a <code>min_chunk_size</code> floor against fragments too small to answer from - ships in the now-sunset <code>langchain-experimental</code> package
SemanticSplitterNodeParser	<code>SemanticSplitterNodeParser(buffer_size=1, breakpoint_percentile_threshold=95, embed_model=embed_model)</code>	- Groups sentences by embedding similarity and breaks where the topic shifts - the actively maintained LlamaIndex counterpart to <code>SemanticChunker</code>
SentenceWindowNodeParser	<code>SentenceWindowNodeParser.from_defaults(window_size=3)</code>	- Splits into one sentence per node, storing the surrounding sentences in metadata - pair with a <code>MetadataReplacementNodePostProcessor</code> to swap in the window before the LLM sees it
HierarchicalNodeParser	<code>HierarchicalNodeParser.from_defaults(chunk_sizes=[2048, 512, 128])</code>	- Chunks a document into several levels of decreasing size, each child linked to its parent - pairs with the <code>AutoMergingRetriever</code> to swap in a parent once most of its children are retrieved
Late Chunking	<code>chunker = LateChunker(embedding_model=long_context_model, chunk_size=512)</code>	- Embeds the whole document first with a long-context model, then splits the token embeddings into chunks - each chunk keeps surrounding context that early chunking would otherwise strip away
chunk_overlap	<code>RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)</code>	Repeats tail tokens/characters across chunks (typically 10-20% of chunk size) to reduce boundary misses
Metadata	<code>Document(page_content=text, metadata={"source": url, "page": 12})</code>	Carries provenance and filters (e.g., doc id, page, section) through retrieval

TABLE 4

Embedding Models

The embedding model is what turns text into the vectors your search runs on, so its quality sets the ceiling on retrieval accuracy. The choice usually comes down to a few trade-offs: proprietary APIs like OpenAI, Cohere, Voyage, and Gemini versus self-hostable open models like BGE-M3, dimension count and cost, and how well each handles non-English or domain-specific corpora.

Model	Example	Description
text-embedding-3-small	<pre>client.embeddings.create(model="text-embedding-3-small", input="...")</pre>	- OpenAI's cost-effective embedding model 1536-dim, supports dimension reduction to 512 with minimal recall loss.
text-embedding-3-large	<pre>client.embeddings.create(model="text-embedding-3-large", input="...")</pre>	- OpenAI's high-accuracy model 3072-dim, supports reduction to 256+ - strong English performance across MTEB.
Cohere embed-v4	<pre>co.embed(texts=[...], model="embed-v4.0", input_type="search_document")</pre>	- Multimodal and multilingual, embeds text, images, and mixed-modality PDFs in one model 1536-dim by default, configurable to 256/512/1024 - strong choice for non-English corpora and mixed-modality search.
Voyage voyage-4-large	<pre>vo.embed(texts, model="voyage-4-large", input_type="document", output_dtype="int8")</pre>	- Voyage AI's current flagship embedding model - natively produces Matryoshka embeddings and supports int8/binary output via <code>output_dtype</code> for cheaper storage - domain-specific variants (code, finance, law) are also offered separately.
Gemini Embedding 2	<pre>client.models.embed_content(model="gemini-embedding-2", contents="...")</pre>	- Google's first multimodal embedding model: text, image, video, audio, and PDF share one vector space 3072-dim by default (Matryoshka-trained; auto-normalizes truncated 768/1536-dim outputs) - sets a new performance standard, outperforming prior models on text, image, and video tasks (April 2026).
BGE-M3	<pre>model = BGEM3FlagModel("BAAI/bge-m3"); model.encode(texts)</pre>	- Open-source - supports dense, sparse, and multi-vector retrieval in one model - self-hostable on a single GPU.
E5-Mistral	<pre>model = SentenceTransformer("float/e5-mistral-7b-instruct")</pre>	- Open-source 7B model - competitive with proprietary models on MTEB instruction-tuned for strong passage retrieval.

TABLE 5

Dense Indexing and ANN

Once you have millions of vectors, scanning all of them per query is too slow: approximate nearest-neighbor (ANN) indexes trade a sliver of recall for orders-of-magnitude faster search. Here you'll find the graph-, partition- and disk-based structures (HNSW, IVFFlat, DiskANN) that power production vector DBs, the exact FAISS baselines, the quantization and dimension-reduction tricks that shrink memory, and the distance metrics that define "similar."

Index	Example	Description
HNSW	<pre>CREATE INDEX ON items USING hnsw (embedding_vector_cosine_ops) WITH (m = 16, ef_construction = 64); SET hnsw.ef_search = 100;</pre>	Graph-based ANN structure, the de-facto standard in production vector DBs for low query latency at high recall - tuned via <code>m</code> (max connections per layer), <code>ef_construction</code> (build-time candidate list size) and <code>ef_search</code> (query-time candidate list size)
IVFFlat	<pre>CREATE INDEX ON items USING ivfflat (embedding_vector_cosine_ops) WITH (lists = 100);</pre>	Inverted-file ANN: partitions vectors into <code>lists</code> and probes a subset at query time - lower build cost than HNSW.

Index	Example	Description
DiskANN	<pre>diskannpy.build_disk_index(data=vectors, distance_metric="l2", index_directory="idx/", complexity=64, graph_degree=32, search_memory_maximum=1.0, build_memory_maximum=8.0, num_threads=8)</pre>	• SSD-resident ANN index: keeps compact search metadata in RAM and streams the rest from disk • built for billion-scale datasets too large to fit in memory.
IndexFlatL2	<pre>index = faiss.IndexFlatL2(d)</pre>	• Exact L2 distance brute-force baseline in FAISS • useful for small corpora or ground-truth evaluation.
IndexFlatIP	<pre>index = faiss.IndexFlatIP(d)</pre>	Exact inner-product search baseline in FAISS.
Product Quantization (PQ)	<pre>index = faiss.IndexPQ(d, m, nbits)</pre>	Splits each vector into m sub-vectors, each quantized with nbits , compressing them into short codes (up to 64x smaller) to cut memory and speed up search.
Scalar Quantization (SQ)	<pre>ScalarQuantization(type=ScalarType.INT8)</pre>	• Quantizes each dimension to 8-bit integers, a 4x storage cut • lighter than PQ with minimal recall loss • widely used in Qdrant.
Binary Quantization	<pre>models.BinaryQuantization(binary=models.BinaryQuantizationConfig(memory=models.Memory.PINNED))</pre>	• Compresses each dimension to a single bit, up to a 32x storage cut • a Hamming-distance-equivalent dot product prefilters candidates, then rescoring against the original vectors recovers most of the lost recall.
Matryoshka Representation Learning (MRL)	<pre>resized = embd_float[:, :256] (truncate a 1024-dim vector)</pre>	• Trains embeddings so a leading prefix of dimensions is itself a valid, slightly-lower-quality embedding • lets you shrink vectors for storage and speed without re-invoking the embedding model; used by OpenAI, Voyage, and Gemini embedding models.
Cosine	<pre>ORDER BY embedding <=> \$1 LIMIT 10</pre>	Common similarity for normalized embeddings (implemented as distance operator in pgvector).
DotProduct	<pre>vectors_config = VectorParams(size=512, distance=Distance.DOT)</pre>	• Inner product similarity, captures both magnitude and direction • equivalent to cosine when vectors are L2-normalized.
Normalization	<pre>faiss.normalize_L2(x)</pre>	Makes cosine similarity retrieval equivalent to dot-product search.
ColBERT / Multi-Vector	<pre>rag = RAGPretrainedModel.from_pretrained("colbert-ir/colbertv2.0"); rag.search(q, k=5)</pre>	• Stores per-token embeddings • scores with MaxSim (max cosine per query token) • near cross-encoder accuracy at near bi-encoder speed.

TABLE 6

Vector Databases

A vector database is where your embeddings actually live and get queried in production, bundling the ANN index with storage, metadata filtering, and operations. The real decision is which one fits your situation: pgvector if you already run Postgres, managed Pinecone for zero-ops scale, Qdrant or Weaviate for open-source flexibility, Chroma for quick prototyping.

Database	Example	Description
pgvector	<pre>CREATE INDEX ON docs USING hnsw (embedding vector_cosine_ops);</pre>	• PostgreSQL extension • keeps vectors and app data in the same table/transaction • the default choice if you already run Postgres, and scales well past 5M vectors when paired with the pgvector scale extension.
Pinecone	<pre>index.upsert(vectors=[(id, emb, meta)]); index.query(vector=q, top_k=10)</pre>	• Fully-managed SaaS • serverless indexes scale automatically, no infrastructure to manage • supports dense + sparse hybrid search in a single index.

Database	Example	Description
Qdrant	<code>client.query_points("docs", query=q, query_filter=Filter(...), limit=10)</code>	• Open-source Rust-native DB • rich payload filtering, scalar/binary quantization, self-hosted or Qdrant Cloud.
Weaviate	<code>collection.query.near_text("...", limit=10)</code>	• Open-source • built-in vectorization modules (auto-embeds raw text), built-in hybrid BM25F+vector search • REST, GraphQL and gRPC APIs.
Milvus / Zilliz	<code>client.search(collection_name="docs", data=[q_emb], limit=10)</code>	• Open-source • GPU-accelerated index types, distributed clusters that scale to tens of billions of vectors • Zilliz Cloud is the managed version.
Chroma	<code>collection.query(query_texts=["..."], n_results=10)</code>	• Embedded or client-server • zero-setup developer experience • best for prototyping and local development.
LanceDB	<code>table.search(q_emb).limit(10).to_arrow()</code>	• Zero-copy columnar storage (Lance format) • embedded/in-process, disk-based indexing for larger-than-RAM datasets.

TABLE 7

Sparse and Hybrid Retrieval

Dense vectors are great at meaning but weak at exact terms, error codes, product names, and rare keywords often slip through. Sparse methods like BM25 catch exactly those, and hybrid retrieval fuses both signals so you don't have to choose; the rest of the rows cover the filtering and diversity controls that shape the final candidate set.

Retriever	Example	Description
HybridSearch	<code>collection.query.hybrid(query="...", alpha=0.5, limit=10)</code>	• Combines sparse and dense vector signals into one ranked list • the <code>alpha</code> weight tunes the blend: 1 is a pure vector search, 0 is a pure keyword search
BM25	$\text{score}(q,d) = \sum_{t \in q} \frac{\text{IDF}(t) \cdot (f(t,d) \cdot (k_1 + 1))}{(f(t,d) + k_1) \cdot (1 - b + b \cdot (d / (\text{avgdl})))}$	• Term-based ranking model used for keyword (sparse) retrieval • strong on exact terms, error codes, and product names.
Learned Sparse Retrieval (SPLADE)	<code>python3 -m splade.index model_type_or_dir=naver/splade-cocondenser-ensembledistil</code> <code>python3 -m splade.retrieve model_type_or_dir=naver/splade-cocondenser-ensembledistil</code>	• Neural model that learns query/document sparse expansion via a BERT MLM head, indexed like BM25 • catches semantic matches beyond exact keywords while staying interpretable
TopK	<code>vectorstore.as_retriever(search_kwargs={"k": 5})</code>	Controls how many candidates are returned from retrieval.
ScoreThreshold	<code>as_retriever(search_type="similarity_score_threshold", search_kwargs={"score_threshold": 0.8})</code>	• Filters results by minimum similarity score before passing context to the LLM • calibrate the cutoff on a labeled set of queries, sweeping thresholds for the precision or recall you need, instead of guessing a number
MMR	$\text{MMR} = (1 - \lambda) \cdot \text{relevance_score} - \lambda \cdot \max(\text{sim_to_selected})$	• Diversifies selected chunks by trading off relevance vs redundancy • avoids returning near-duplicate passages.
MetadataFiltering	<code>Filter.by_property("source").equal("handbook")</code>	Restricts candidates to documents matching structured metadata predicates before vector search.
Similarity (BM25 config)	<code>"similarity": {"default": {"type": "BM25", "k1": 1.2, "b": 0.75}}</code>	Configures per-field BM25 scoring parameters in Elasticsearch.

TABLE 8

Query Transformation

Users rarely phrase questions the way your documents are written, and a single raw query often misses relevant chunks. These techniques reshape the query before it ever hits the index: rewriting it, expanding it into several variants, generating a hypothetical answer to search with, or breaking a complex question into sub-questions, to close the gap between how people ask and how content is stored.

Technique	Example	Description
QueryRewrite	"Rewrite the question for search: ..."	Converts user input into a search-optimized query string to improve vector recall.
History-Aware Retriever	<pre>from langchain_classic.chains import create_history_aware_retriever chain = create_history_aware_retriever(llm, retriever, rephrase_prompt)</pre>	- Condenses a follow-up question and chat history into a standalone search query - passes the input straight through when there is no history.
MultiQueryRetriever	<pre>from langchain_classic.retrievers.multi_query import MultiQueryRetriever mqr = MultiQueryRetriever.from_llm(retriever, llm)</pre>	Uses an LLM to generate multiple query variants and unions retrieved docs for better recall.
HyDE	<pre>hyp = llm("Write a passage answering: ...") docs = retriever.invoke(hyp)</pre>	- Retrieves using embeddings of a hypothetical document generated from the query - helps with vague or sparse questions.
SelfQueryRetriever	<pre>from langchain_classic.retrievers.self_query.base import SelfQueryRetriever sq = SelfQueryRetriever.from_llm(llm, vectorstore, document_contents, metadata_field_info)</pre>	Uses an LLM to translate natural language into a structured query + metadata filters .
Decomposition	<pre>subqs = ["...", "..."]</pre>	Splits a complex question into sub-questions that are answered individually and combined.
Step-Back Prompting	<pre>broader = llm(f"What is the more general question behind: '{q}'") docs = retriever.invoke(broader)</pre>	- First retrieves for a broader abstraction of the question before the specific query - improves recall on specific questions.
Routing	<pre>from llama_index.core.query_engine import RouterQueryEngine qe = RouterQueryEngine(selector=PydanticSingleS elector.from_defaults(), query_engine_tools=[list_tool, vector_tool])</pre>	- Selects a retriever/index/tool based on query intent or domain - avoids wasting retrieval on the wrong corpus.
Expansion	<pre>q' = q + synonyms(q)</pre>	Adds related terms/phrases to improve recall in sparse retrieval.

TABLE 9

Reranking and Fusion

First-pass retrieval is tuned for speed and recall, so it usually pulls back more than you need with the best chunk not always on top. A reranker does a second, more expensive pass that scores each query-document pair jointly and reshuffles for precision. Anthropic measured reranking cutting retrieval failures by 67% when combined with contextual retrieval.

Ranker	Example	Description
Cross-encoder reranking	<pre>model = CrossEncoder("cross-encoder/ms- marco-MiniLM-L6-v2") scores = model.predict([(q, d) for d in docs])</pre>	- Scores each query-document pair jointly with full attention for higher-precision top-k - compute cost is small vs LLM call.

Ranker	Example	Description
Cohere Rerank	<pre>co.rerank(model="rerank-v4.0-pro", query=q, documents=texts, top_n=10)</pre>	• API reranker producing a relevance-ordered list with scores • Rerank 4 (Dec 2025) supports 100+ languages with a 32k token context window, four times larger than Rerank 3.5.
BGE Reranker	<pre>reranker = FlagReranker("BAAI/bge-reranker-v2-m3"); scores = reranker.compute_score([(q,d) for d in docs])</pre>	• Open-source cross-encoder reranker • multilingual, competitive with commercial rerankers • self-hostable.
LLMRerank	<pre>reranker = LLMRerank(llm=llm, choice_batch_size=5, top_n=3) engine = index.as_query_engine(node_postprocessors=[reranker])</pre>	• Asks an LLM to score and pick the relevant nodes (listwise) • more accurate than a cross-encoder but adds an LLM call per query.
Reciprocal Rank Fusion (RRF)	$RRF(d) = \sum_i 1 / (k + rank_i(d))$	• Fuses multiple ranked lists without needing calibrated similarity scores • $k=60$ is a common default.
EnsembleRetriever	<pre>from langchain_classic.retrievers.ensemble import EnsembleRetriever ens = EnsembleRetriever(retrievers=[r1, r2], weights=[0.5, 0.5])</pre>	Combines multiple retrievers and applies rank fusion to merge results.
ContextualCompressionRetriever	<pre>from langchain_classic.retrievers.contextual_compression import ContextualCompressionRetriever cc = ContextualCompressionRetriever(base_retriever=r, base_compressor=compressor)</pre>	Retrieves then compresses documents to only the query-relevant parts, reducing context noise.
Deduplication (Embeddings RedundantFilter)	<pre>filt = EmbeddingsRedundantFilter(embeddings=embeddings) docs = filt.transform_documents(docs)</pre>	Drops near-duplicate chunks by comparing their embeddings before the LLM sees them.
LongContextReorder	<pre>reorder = LongContextReorder() engine = index.as_query_engine(node_postprocessors=[reorder])</pre>	• Moves the highest-scored nodes to the start and end of the context, away from the middle • counters lost-in-the-middle when a large top-k is retrieved.
MetadataReplacementPostProcessor	<pre>MetadataReplacementPostProcessor(target_metadata_key="window")</pre>	Swaps a matched sentence back for its surrounding window , pairing with sentence-window retrieval.
TopN	<pre>top_n=10</pre>	Limits reranker output to the highest-scoring items only.

TABLE 10

Query Engines and Answer Grounding

This is the generation half of RAG: the components that take retrieved context and turn it into a written answer. Grounding is the key idea: instructing the model to answer only from the provided evidence is the core defense against hallucination, and the citation engines tie each claim back to its source so readers can verify it.

Concept	Example	Description
Grounding	<pre>"Answer only using the provided context."</pre>	• Forces the LLM to base claims on retrieved evidence rather than latent knowledge • core anti-hallucination mechanism.

Concept	Example	Description
Fallback Instruction	"If the context doesn't answer the question, reply: 'The provided documents don't contain sufficient information to answer this question.'"	- Tells the model what to say when retrieved context doesn't cover the query - stops it from guessing an answer.
RetrieverQueryEngine	<pre>qe = RetrieverQueryEngine.from_args(retriever=retriever) resp = qe.query("...")</pre>	LlamaIndex query engine that retrieves nodes then synthesizes a response .
PromptTemplate	<pre>prompt = ChatPromptTemplate.from_messages([("system", "..."), ("human", "{question}")])</pre>	Parameterizes prompts so retrieval context and user input can be inserted reliably.
ResponseMode	<pre>response_mode=ResponseMode.COMPACT</pre>	- Selects how retrieved chunks become the answer: refine (chunk by chunk), compact (default, fewer LLM calls), or tree_summarize (recursive) - set via the <code>response_mode</code> kwarg.
CitationQueryEngine	<pre>qe = CitationQueryEngine.from_args(index=index) resp = qe.query("...")</pre>	Generates answers with inline source citations anchored to retrieved chunks.
Citation Chunk Size	<pre>citation_chunk_size=512</pre>	Sets the chunk size used to form citation units for per-source attribution.
Label Sources	<pre>### Internal KB {kb_chunks} ### Web Search {web_chunks}</pre>	- Tags each context block with where it came from (internal corpus vs. web-search fallback) - lets the model attribute or prioritize a claim by its source.
Streaming	<pre>stream=True</pre>	- Streams partial tokens while a completion is being generated - reduces perceived latency.
CondensePlusContextChatEngine	<pre>chat_engine = CondensePlusContextChatEngine.from_defaults(retriever=retriever) chat_engine.chat("...")</pre>	- Chat engine that condenses the conversation into a standalone question, retrieves context for it, then answers - keeps multi-turn RAG chats on topic.
Context Window	<pre># 200k-token context window = input + output combined client.messages.create(model="claude-sonnet-4-5", max_tokens=1024, ...)</pre>	- The total tokens (prompt, retrieved context and response) a model can hold in one request separate from <code>max_tokens</code>, which only caps the output length.

TABLE 11

Advanced RAG Architectures

Beyond the basic retrieve-then-generate loop sit the patterns that define modern production RAG. Most share one move: letting the system decide how to retrieve rather than hard-coding a single pass. Agentic RAG treats retrieval as a tool the LLM calls, corrective and self-RAG grade their own evidence, multi-hop chains several retrievals, and GraphRAG retrieves over a knowledge graph instead of flat chunks.

Pattern	Example	Description
Agentic RAG	<pre>tools=[search_knowledge_base]; agent.run(question)</pre>	- LLM decides when, what, and how to retrieve as a tool call - handles multi-step and conditional retrieval needs.
Corrective RAG (CRAG)	<pre>grade = evaluator.score(doc, q); if grade == "incorrect": web_search(q)</pre>	Lightweight evaluator grades each retrieved document (correct/ambiguous/incorrect) and triggers web search on failures.
Self-RAG	<pre># model uses reflection tokens: [Retrieve], [ISREL], [ISSUP]</pre>	Trains a single LM to adaptively retrieve on-demand and self-critique retrieved passages and its own generations.

Pattern	Example	Description
Hybrid RAG	<pre>docs = retrieve(rewrite(q)) if not relevant(docs): docs = retrieve(refine(q)) answer = generate(docs); validate(answer)</pre>	- LangChain's middle ground between 2-Step and Agentic RAG - adds query enhancement, retrieval validation, and answer validation around an otherwise fixed pipeline.
GraphRAG	<pre>graphrag.query(query_type="local", query="...")</pre>	Microsoft's approach: extracts a knowledge graph from the corpus, builds community summaries, and retrieves via local, global, or DRIFT (community-informed local) search.
Property Graph Index	<pre>index = PropertyGraphIndex.from_documents(docs) retriever = PGRetriever([LLMSynonymRetriever(index), VectorContextRetriever(index)])</pre>	- LlamaIndex's knowledge-graph index: labeled nodes and relations extracted from chunks - queries combine sub-retrievers (synonym lookup, vector similarity, text-to-Cypher) instead of one flat index.
Multi-Hop RAG	<pre>for hop in range(MAX_HOPS): docs=retrieve(q); q=refine(q,docs)</pre>	- Chains multiple retrieval steps where each hop's results inform the next query - needed for questions that require reasoning over several pieces of supporting evidence.
Contextual Retrieval	<pre>chunk = f"{llm_context(doc, chunk)}\n\n{chunk}"; embed(chunk)</pre>	- Anthropic technique: prepends chunk-specific context before embedding and BM25 indexing - reduces retrieval failures by 49-67% combined with reranking.
RAPTOR	<pre>pack = RaptorPack(documents, llm=llm, embed_model=embed_model) nodes = pack.run("query", mode="collapsed")</pre>	Recursively embeds, clusters, and summarizes chunks into a tree from the bottom up - retrieves across levels of abstraction instead of only leaf chunks.
Sentence-Window Retrieval	<pre>parser = SentenceWindowNodeParser.from_defaults(window_size=3)</pre>	- Parses documents into single-sentence nodes for precise embedding matches - a postprocessor swaps the matched sentence back for its surrounding window before synthesis.
Auto-Merging Retriever	<pre>retriever = AutoMergingRetriever(base_retriever, storage_context)</pre>	- Recursively merges leaf nodes into their parent when enough of a parent's children are retrieved - consolidates fragmented small chunks into one larger context.
MCP-Based Retrieval	<pre>tool_list = await client.list_tools() agent.run(question, tools=tool_list.tools)</pre>	- Exposes a knowledge base as a Model Context Protocol server so any MCP-compatible agent can call retrieval without bespoke integration code - the standard way production agentic RAG systems wire retrieval into a tool-calling loop.
Adaptive RAG	<pre>route = classifier(query); pipeline = routes[route]</pre>	A query complexity classifier routes each query to the appropriate pipeline (no retrieval, single-hop, or multi-hop), saving cost on simple queries.
Long-Context vs RAG	<pre>if tokens(corpus) <= model.context_window: answer = llm(corpus + question) else: answer = rag_pipeline(question)</pre>	Long-context prompting can outperform RAG on question answering over a corpus that fits the window, but RAG keeps an edge on dialogue and general queries at lower per-call cost.

TABLE 12

Storage, Persistence, and Caching

Vectors, full documents and index structures get persisted here so a RAG system doesn't rebuild them on every restart. At scale, the caching and incremental-indexing rows matter most: re-embedding only changed documents and reusing cached embeddings or answers are where the real cost savings come from.

Store	Example	Description
VectorStore	<code>vector_store.persist(persist_path="./storage/vector_store.json")</code>	Persists embedding vectors (and often the source chunks) so retrieval doesn't require re-embedding after a restart.
DocStore	<code>docstore.persist()</code>	Stores full documents (Node objects) separately from the vector and index stores, for reload without re-parsing sources.
IndexStore	<code>index_store.persist(persist_path="./storage/index_store.json")</code>	Persists index metadata and structures for reload without a full rebuild.
StorageContext	<code>StorageContext.from_defaults(persist_dir="./storage")</code>	Bundles the docstore, index store and vector store into one container an index or query pipeline persists and loads together.
Incremental Indexing	<code>if doc.hash != stored.hash: re_embed(doc)</code>	Re-indexes only changed documents to keep the vector store current without full rebuilds.
Record Manager	<code>from langchain_classic.indexes import SQLRecordManager index(docs, record_manager, vectorstore, cleanup="incremental")</code>	- LangChain's indexing API tracks document hashes and write times so unchanged content is skipped <code>SQLRecordManager</code> now lives in langchain-classic.
EmbeddingCache	<code>CacheBackedEmbeddings.from_bytes_store(underlying_embedder, store)</code>	- Avoids recomputing embeddings for identical text by hashing it as the cache key <code>CacheBackedEmbeddings</code> now lives in langchain-classic.
Semantic Cache	<code>cache.init(embedding_func=onnx.to_embeddings, similarity_evaluation=SearchDistanceEvaluation())</code>	Returns a stored answer when a new query's embedding is similar enough to one already answered, cutting repeat LLM calls.
Namespace	<code>index.query(namespace="prod", vector=q, top_k=10)</code>	Separates tenant or environment data within one vector index for multi-tenant isolation.
TTL	<code>EXPIRE cache:answer:<hash> 3600</code>	Expires cached retrieval or generation artifacts automatically after a time window.

TABLE 13

RAG Evaluation Metrics and Frameworks

RAG quality splits into two questions: did retrieval surface the right context, and did generation stay faithful to it. These metrics (faithfulness, context precision and recall, response relevancy) pin down each side, and the frameworks below run them in CI against a golden dataset so a pipeline change can be graded before it ships.

Metric	Example	Description
Faithfulness	<code>faithfulness ∈ [0,1]</code>	- Measures consistency with retrieved context - the primary anti-hallucination metric.
Response Relevancy (Answer Relevancy)	<code>response_relevancy ∈ [0,1]</code>	Measures how well the answer addresses the question .
Context Precision	<code>context_precision ∈ [0,1]</code>	Measures whether retrieved contexts are useful for answering the question.
Context Recall	<code>context_recall ∈ [0,1]</code>	Measures how much of the needed information is present in retrieved contexts.

Metric	Example	Description
Retrieval Ranking Metrics	<code>RetrieverEvaluator.from_metric_names(["hit_rate", "mrr", "ndcg", "recall"], retriever=retriever)</code>	- Classic information-retrieval metrics scored against ground-truth doc IDs - Hit Rate and MRR check if/where the first relevant chunk lands; nDCG and Recall@k grade the whole ranked list
Factual Correctness	<code>factual_correctness ∈ [0,1]</code>	Measures whether the answer is factually correct against a reference.
Noise Sensitivity	<code>noise_sensitivity ∈ [0,1]</code>	Measures robustness to irrelevant context : does the answer degrade when noisy chunks are included?
Context Entities Recall	<code>context_entities_recall ∈ [0,1]</code>	Measures entity recall over retrieved context vs reference.
Aspect Critic	<code>AspectCritic(name="maliciousness", definition="Is the submission harmful?")</code>	LLM-judge style metric for assessing a specific aspect of the output (e.g., conciseness, harmlessness).
LLM-as-a-Judge Biases	<pre>swap(A, B) score_ab = judge(A, B) score_ba = judge(B, A) # check position bias</pre>	- Using an LLM to score outputs instead of a human, the basis of Faithfulness, Aspect Critic and G-Eval style metrics - prone to position, verbosity and self-enhancement biases, so swap answer order and cross-check with a second judge model
Golden Dataset	<code>[{"question": q, "ground_truth": a, "source_docs": []}]</code>	- Curated Q&A set with approved answers and source documents - used to benchmark pipeline changes before deployment.
Synthetic Test Set Generation	<pre>generator = TestsetGenerator(llm=generator_llm, embedding_model=embeddings) testset = generator.generate_with_langchain_docs(docs, testset_size=10)</pre>	- Builds a question, answer and context testset automatically from a document corpus via a knowledge graph - scales evaluation coverage before a human reviews the generated queries
DeepEval	<pre>from deepeval import assert_test answer_relevancy_metric = AnswerRelevancyMetric(threshold=0.7) assert_test(test_case, [answer_relevancy_metric])</pre>	pytest-compatible LLM testing framework with metrics for RAG, agents and multi-turn chats - runs via <code>deepeval test run</code>, built for CI/CD quality gates.
Arize Phoenix	<pre>pip install arize-phoenix && phoenix serve # instrument your app, then run evals against the traces</pre>	- Open-source AI observability platform - OpenTelemetry-based tracing with built-in RAG evaluators - self-hostable.
LangSmith	<pre>from langsmith.evaluation import evaluate results = evaluate(predict, data="rag-eval", evaluators=[accuracy])</pre>	- LangChain-native tracing and evaluation platform - deep visibility into chain execution steps and LLM calls.

TABLE 14

Observability and Security

Once RAG is live you need to see inside it and defend it. The observability rows cover the OpenTelemetry GenAI spans and tracing tools that expose retrieval counts, token usage, and latency for debugging. The security rows map the OWASP LLM risks that hit RAG specifically: prompt injection riding in through retrieved context, the dedicated vector-and-embedding-weakness category covering cross-tenant leakage and embedding inversion, and corpus poisoning.

Concept	Example	Description
GenAI spans	<code>gen_ai.operation.name = "chat"</code>	Standardizes tracing semantics for GenAI operations (inference, retrieval, tools) via OpenTelemetry.
Retrieval span	<code>gen_ai.retrieval.top_k = 5</code>	Captures retrieval metadata (requested top_k, per-document relevance scores) to debug relevance vs latency tradeoffs.

Concept	Example	Description
Token metrics	<code>gen_ai.client.inference.usage.input_tokens</code> <code>gen_ai.client.inference.usage.output_tokens</code>	Counters that record token usage to monitor cost and performance over time.
Langfuse	<code>client = get_client()</code> <code>with client.start_as_current_observation(name="rag-query") as span:</code> <code>...</code>	Open-source LLM observability platform with trace-based debugging, evals, and a prompt management UI.
Prompt Injection	"Ignore previous instructions and ..."	Attacker-controlled input attempts to override system/developer intent via the retrieval context.
Vector and Embedding Weaknesses	"resume with hidden white-on-white text: ignore prior instructions..."	- OWASP's RAG-specific risk category: unauthorized cross-tenant retrieval, embedding-inversion attacks that recover source text, and hidden instructions smuggled inside indexed documents - mitigated with permission-aware vector stores and input validation before indexing.
Access Control Inheritance	"Tag each chunk with <code>allowed_roles</code> ; drop retrieved chunks the caller's roles don't include"	Carries document-level permissions (classification, department, tenant) through to every vector chunk and re-checks them at query time , not just at ingestion.
Sensitive Information Disclosure	"Print your hidden prompt"	Leakage of secrets, system prompts, or private data via model outputs.
Redact PII	"Redact PII, secrets, and credentials from generated answers, scoped to the caller's access level"	Strips personal, secret, or regulated data from generated answers before they reach the user.
Data and Model Poisoning	"Upload malicious docs into the KB"	Corrupts the retrieval corpus or training data so the model is grounded in incorrect or malicious context.
Improper Output Handling	<code>render_html(llm_output)</code>	Treating model output as trusted without validation or sanitization can lead to downstream injection or execution.
Unbounded Consumption	"Summarize 10MB of text"	- Uncontrolled inference volume or oversized inputs drive excessive compute/cost, denial of service, or model/IP theft - mitigated with rate limits, input-size caps, and timeouts.

TABLE 15

Multimodal RAG

A huge share of real knowledge lives in PDFs full of tables, charts, and scanned layouts that text extraction mangles. Multimodal RAG sidesteps that by treating document pages as images: ColPali-style retrievers embed page images directly and a vision-language model reads them, increasingly replacing the old OCR-then-embed pipeline for complex documents.

Technique	Example	Description
ColPali	<code>model = ColPali.from_pretrained("vidore/colpali-v1.3").eval()</code> <code>processor = ColPaliProcessor.from_pretrained("vidore/colpali-v1.3")</code> <code>embeddings =</code> <code>model(**processor.process_images(images))</code>	- VLM-based retriever that produces multi-vector embeddings directly from document page images via late interaction - no OCR needed

Technique	Example	Description
ColQwen2	<pre>model = ColQwen2.from_pretrained("vidore/colqwen2-v1.0").eval() processor = ColQwen2Processor.from_pretrained("vidore/colqwen2-v1.0") scores = processor.score_multi_vector(query_embeddings, image_embeddings)</pre>	- ColPali-style retriever built on Qwen2-VL-2B instead of PaliGemma, taking dynamic image resolutions rather than a fixed resize - the backbone most current multimodal-RAG tooling ships by default
Byaldi	<pre>RAG = RAGMultiModalModel.from_pretrained("vidore/colqwen2-v1.0") RAG.index(input_path="docs/", index_name="my_index") results = RAG.search(query, k=3)</pre>	Python library wrapping ColPali-family models with a familiar API for indexing PDFs and searching by visual content
MultiVectorRetriever	<pre>retriever = MultiVectorRetriever(vectorstore=vs, docstore=store, id_key="doc_id") retriever.vectorstore.add_documents(table_summary_docs) retriever.docstore.mset([(doc_id, raw_table)])</pre>	- Splits a PDF into text, table, and image elements, indexing a short summary of each while storing the raw element in a docstore - the LLM sees the original table or image at synthesis time
VLM Synthesis	<pre>inputs = vl_model_processor(text=[query], images=retrieved_pages, return_tensors="pt").to("cuda") generated_ids = vl_model.generate(**inputs, max_new_tokens=500)</pre>	Uses a Vision Language Model (e.g., Qwen2-VL, GPT-4V) to answer based on retrieved document page images
OCR Pipeline	<pre>text = ocr_engine.extract(page_image); embed(text)</pre>	- Extracts text from scanned pages via OCR (e.g., Tesseract) before chunking and embedding - increasingly displaced by ColPali-style retrievers for documents with complex layouts, tables, and figures

References

Official Documentation

- 2-Step RAG vs Agentic RAG
- Ingestion
- Chunking
- Embedding
- Indexing
- Reranking
- Synthesis
- Citations
- Evaluation
- SimpleDirectoryReader
- Document and Node
- Document Loaders (LangChain)
- Data Connectors (LlamaHub Readers)
- IngestionPipeline
- Metadata Extractors
- LlamaParse
- SentenceSplitter
- MarkdownHeaderTextSplitter
- HTMLHeaderTextSplitter
- CodeSplitter
- ParentDocumentRetriever
- SemanticSplitterNodeParser
- SentenceWindowNodeParser
- HierarchicalNodeParser
- chunk_overlap
- Metadata
- Cohere embed-v4
- Voyage voyage-4-large
- Gemini Embedding 2
- DiskANN
- Scalar Quantization (SQ)
- Binary Quantization
- Matryoshka Representation Learning (MRL)
- Qdrant
- Weaviate
- Milvus / Zilliz
- Chroma
- LanceDB
- HybridSearch
- TopK
- MMR
- MetadataFiltering
- Similarity (BM25 config)
- QueryRewrite
- History-Aware Retriever
- MultiQueryRetriever
- SelfQueryRetriever
- Routing
- Cross-encoder reranking
- Cohere Rerank
- EnsembleRetriever
- ContextualCompressionRetriever

53. Deduplication (EmbeddingsRedundantFilter)**55. Grounding****57. RetrieverQueryEngine****59. CitationQueryEngine****61. CondensePlusContextChatEngine****63. Hybrid RAG****65. Sentence-Window Retrieval****67. MCP-Based Retrieval****69. DocStore****71. StorageContext****73. Record Manager****75. Namespace****77. Faithfulness****79. Context Precision****81. Retrieval Ranking Metrics****83. Noise Sensitivity****85. Aspect Critic****87. DeepEval****89. Langfuse****91. ParentDocumentRetriever****93. Pinecone****95. Milvus / Zilliz****97. Cohere Rerank****99. Namespace****101.** https://docs.llamaindex.ai/en/stable/module_guides/deploying/query_engine/**103.** https://docs.llamaindex.ai/en/stable/examples/query_engine/sub_question_query_engine/**105.** https://python.langchain.com/docs/how_to/self_query/**107.** https://python.langchain.com/docs/concepts/prompt_templates/**109.** https://python.langchain.com/docs/how_to/vectorstore_retriever/**111.** <https://platform.openai.com/docs/guides/text-generation>**113.** <https://qdrant.tech/documentation/>**115.** <https://docs.weaviate.io/>**117.** <https://docs.trychroma.com/>**119.** <https://docs.confident-ai.com/>**121.** <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>**123.** <https://opentelemetry.io/docs/specs/semconv/registry/attributes/gen-ai/>**125.** <https://supabase.com/docs/guides/database/extensions/pgvector>**127.** <https://www.sbert.net/docs/cross-encoder.html>**54. TopN****56. Fallback Instruction****58. PromptTemplate****60. Streaming****62. Context Window****64. Property Graph Index****66. Auto-Merging Retriever****68. VectorStore****70. IndexStore****72. Incremental Indexing****74. EmbeddingCache****76. TTL****78. Response Relevancy (Answer Relevancy)****80. Context Recall****82. Factual Correctness****84. Context Entities Recall****86. Synthetic Test Set Generation****88. Arize Phoenix****90. OCR Pipeline****92. Cohere embed-v4****94. Weaviate****96. ScoreThreshold****98. Record Manager****100.** https://docs.llamaindex.ai/en/stable/module_guides/loading/documents_and_nodes/**102.** https://docs.llamaindex.ai/en/stable/api_reference/postprocessor/similarity/**104.** https://python.langchain.com/docs/how_to/semantic-chunker/**106.** https://python.langchain.com/docs/how_to/routing/**108.** https://python.langchain.com/docs/concepts/text_splitters/**110.** <https://platform.openai.com/docs/guides/prompt-engineering>**112.** <https://docs.pinecone.io/>**114.** <https://docs.weaviate.io/weaviate/concepts/data>**116.** <https://milvus.io/docs/overview.md>**118.** https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/aspect_critic/**120.** <https://docs.smith.langchain.com/>**122.** <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-metrics/>**124.** <https://redis.io/docs/latest/develop/data-types/strings/>**126.** <https://www.mongodb.com/docs/atlas/ai-integrations/langchain/parent-document-retrieval/>**128.** https://dev.to/young_gao/rag-is-not-dead-advanced-retrieval-patterns-that-actually-work-in-2026-2gbo**Specifications & Standards****129. Prompt Injection****131. Access Control Inheritance****133. Data and Model Poisoning****135. Unbounded Consumption****137.** <https://genai.owasp.org/llm-top-10/>**139.** <https://genai.owasp.org/llmrisk/llm04-model-denial-of-service/>**141.** <https://genai.owasp.org/llmrisk/llm08-excessive-agency/>**Academic Papers****142. Retrieval-Augmented Generation (RAG)****144. HyDE****146. Step-Back Prompting****148. Corrective RAG (CRAG)****150. Multi-Hop RAG****130. Vector and Embedding Weaknesses****132. Sensitive Information Disclosure****134. Improper Output Handling****136.** <https://owasp.org/www-project-top-10-for-large-language-model-applications/>**138.** <https://genai.owasp.org/llmrisk/llm02-insecure-output-handling/>**140.** <https://genai.owasp.org/llmrisk/llm05-supply-chain-vulnerabilities/>**143. HNSW****145. Decomposition****147. Agentic RAG****149. Self-RAG****151. RAPTOR**

- 152. [Adaptive RAG](#)
- 154. [LLM-as-a-Judge Biases](#)
- 156. <https://arxiv.org/pdf/2005.11401>
- 158. <https://arxiv.org/abs/2409.04701>
- 160. <https://arxiv.org/abs/2511.00444>
- 162. <https://dl.acm.org/doi/10.1145/290941.291025>
- 164. <https://openreview.net/forum?id=ogjBpZ8uSi>

Source Code & Repositories

- 166. [Unstructured \(partition_pdf\)](#)
- 168. [TokenTextSplitter](#)
- 170. [SemanticChunker](#)
- 172. [Product Quantization \(PQ\)](#)
- 174. [ColBERT / Multi-Vector](#)
- 176. [Semantic Cache](#)
- 178. [Token metrics](#)
- 180. [Byaldi](#)
- 182. <https://github.com/langchain-ai/langchain>
- 184. <https://github.com/facebookresearch/faiss>
- 186. <https://github.com/PranavGovindu/Self-Corrective-Agentic-RAG>
- 188. <https://github.com/microsoft/multi-modal-rag-with-colpali>
- 153. [Long-Context vs RAG](#)
- 155. [OCR Pipeline](#)
- 157. <https://arxiv.org/pdf/2212.10496>
- 159. <https://arxiv.org/pdf/2409.04701>
- 161. <https://dl.acm.org/doi/10.1145/1571941.1572114>
- 163. <https://aclanthology.org/2023.acl-long.99/>
- 165. <https://openreview.net/forum?id=74QmBTv0Zf>
- 167. [RecursiveCharacterTextSplitter](#)
- 169. [RecursiveJsonSplitter](#)
- 171. [IVFFlat](#)
- 173. [Normalization](#)
- 175. [Learned Sparse Retrieval \(SPLADE\)](#)
- 177. [GenAI spans](#)
- 179. [ColPali](#)
- 181. [SemanticChunker](#)
- 183. https://github.com/run-llama/llama_index
- 185. <https://github.com/microsoft/graphrag>
- 187. <https://github.com/jina-ai/late-chunking>

Articles & Tutorials

- 189. [Late Chunking](#)
- 191. [E5-Mistral](#)
- 193. [IndexFlatIP](#)
- 195. [Pinecone](#)
- 197. [Expansion](#)
- 199. [Reciprocal Rank Fusion \(RRF\)](#)
- 201. [Contextual Retrieval](#)
- 203. [ColQwen2](#)
- 205. [VLM Synthesis](#)
- 207. [Pinecone](#)
- 209. [DeepEval](#)
- 211. https://python.langchain.com/api_reference/text_splitters/character/langchain_text_splitters.character.RecursiveCharacterTextSplitter.html
- 213. https://python.langchain.com/api_reference/text_splitters/base/langchain_text_splitters.base.TokenTextSplitter.html
- 215. https://python.langchain.com/api_reference/text_splitters/html/langchain_text_splitters.html.HTMLHeaderTextSplitter.html
- 217. https://python.langchain.com/api_reference/langchain/retrievers/langchain_retrievers.contextual_compression.ContextualCompressionRetriever.html
- 219. https://python.langchain.com/api_reference/langchain/retrievers/langchain_retrievers.parent_document_retriever.ParentDocumentRetriever.html
- 221. https://faiss.ai/cpp_api/struct/structfaiss_1_1IndexPQ.html
- 223. <https://qdrant.tech/articles/binary-quantization-openai/>
- 225. <https://phoenix.arize.com/>
- 227. <https://microsoft.github.io/graphrag/query/overview/>
- 229. <https://modelcontextprotocol.io/introduction>
- 231. <https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/>
- 233. <https://www.postgresql.org/about/news/pgvector-080-released-2952/>
- 235. <https://redis.io/blog/get-better-rag-responses-with-ragas/>
- 237. <https://www.datadoghq.com/blog/llm-otel-semantic-convention/>
- 239. <https://blog.prem.ai/rag-chunking-strategies-the-2026-benchmark-guide/>
- 190. [BGE-M3](#)
- 192. [IndexFlatL2](#)
- 194. [DotProduct](#)
- 196. [BM25](#)
- 198. [BGE Reranker](#)
- 200. [GraphRAG](#)
- 202. [LangSmith](#)
- 204. [MultiVectorRetriever](#)
- 206. [Gemini Embedding 2](#)
- 208. [Milvus / Zilliz](#)
- 210. https://python.langchain.com/api_reference/core/retrievers/langchain_core_retrievers.BaseRetriever.html
- 212. https://python.langchain.com/api_reference/text_splitters/character/langchain_text_splitters.character.CharacterTextSplitter.html
- 214. https://python.langchain.com/api_reference/text_splitters/markdown/langchain_text_splitters.markdown.MarkdownHeaderTextSplitter.html
- 216. https://python.langchain.com/api_reference/langchain/retrievers/langchain_retrievers.ensemble.EnsembleRetriever.html
- 218. https://python.langchain.com/api_reference/langchain/retrievers/langchain_retrievers.multi_query.MultiQueryRetriever.html
- 220. <https://faiss.ai/index.html>
- 222. https://faiss.ai/cpp_api/namespace/namespacefaiss.html
- 224. <https://lancedb.github.io/lancedb/>
- 226. <https://langfuse.com/docs>
- 228. <https://www.microsoft.com/en-us/research/project/graphrag/>
- 230. <https://cohere.com/blog/rerank-4>
- 232. <https://weaviate.io/blog/hybrid-search-explained>
- 234. <https://aws.amazon.com/blogs/database/optimize-generative-ai-applications-with-pgvector-indexing-a-deep-dive-into-ivfflat-and-hnsw-techniques/>
- 236. <https://qdrant.tech/blog/rag-evaluation-guide/>
- 238. <https://www.sbert.net/examples/applications/cross-encoder/README.html>
- 240. <https://www.prem.ai/blog/rag-chunking-strategies-the-2026-benchmark-guide/>

241. <https://blog.prem.ai/rag-evaluation-metrics-frameworks-testing-2026/>
243. <https://www.firecrawl.dev/blog/best-chunking-strategies-rag>
245. <https://callsphere.tech/blog/rag-architecture-patterns-2026-retrieval-augmented-generation>
247. <https://encore.dev/articles/best-vector-databases>
249. <https://www.lancedb.com/blog/modified-rag-parent-document-bigger-chunk-retriever-62b3d1e79bc6>
251. <https://graphrag.com/concepts/intro-to-graphrag/>
253. <https://blog.gopenai.com/the-fidelity-crisis-in-rag-why-late-interaction-colbert-is-the-4k-image-of-search-vs-e978d96b25b8>
255. <https://deepeval.com/blog/top-5-llm-evaluation-frameworks>
257. <https://rthesis.ai/post/best-llm-evaluation-testing-tools>
259. <https://blog.starmorph.com/blog/rag-techniques-compared-best-practices-guide>
261. <https://huggingface.co/blog/manu/colpali>
263. <https://www.dev.to/polliog/postgresql-as-a-vector-database-when-to-use-pgvector-vs-pinecone-vs-weaviate-4kfi>
265. <https://www.aws.amazon.com/blogs/machine-learning/contextual-retrieval-in-anthropic-using-amazon-bedrock-knowledge-bases/>
267. <https://pecollective.com/tools/best-embedding-models/>
242. <https://www.prem.ai/blog/best-embedding-models-for-rag-2026-ranked-by-mteb-score-cost-and-self-hosting/>
244. <https://www.firecrawl.dev/blog/best-vector-databases>
246. <https://rapidclaw.dev/blog/rag-architecture-ai-agents-guide-2026>
248. <https://procogia.com/unlocking-rags-potential-mastering-advanced-techniques-part-1/>
250. <https://www.meilisearch.com/blog/graph-rag>
252. <https://weaviate.io/blog/late-interaction-overview>
254. <https://deepeval.com/blog/deepeval-vs-trulens>
256. <https://www.getmaxim.ai/articles/the-5-best-rag-evaluation-tools-you-should-know-in-2026/>
258. <https://ragflow.io/blog/rag-review-2025-from-rag-to-context>
260. <https://www.turingpost.com/p/ragtypes>
262. <https://milvus.io/blog/choose-embedding-model-rag-2026.md>
264. <https://ackuity.ai/blog/mitigating-vector-and-embedding-weaknesses>
266. <https://nemorize.com/roadmaps/2026-modern-ai-search-rag-roadmap/lessons/agent-rag-systems>
268. <https://pecollective.com/tools/text-embedding-models-compared/>

Video

269. <https://www.youtube.com/watch?v=j66Db1SB1YY>
271. <https://www.youtube.com/watch?v=VfjIYjYFVt4>
273. <https://www.youtube.com/watch?v=-1zMU1a625E>
275. <https://www.youtube.com/watch?v=buzWGXOydD8>
270. https://www.youtube.com/watch?v=nMCII_xtUbw
272. <https://www.youtube.com/watch?v=0fackgiKTiA>
274. <https://www.youtube.com/watch?v=vT-DpLv29Q>